

Advanced SQL

— Chapter 5

2025 年 10
月

School of Computer Science



北京邮电大学

Outline

- 1. Accessing SQL from a programming language
- 2. Functions and procedures
- 3. Triggers

5.1 Accessing SQL from a Programming Language

- DBAS = users + application programs + DBMS + DB
- API (application program interface) to **interact** with DB server
- Application **conducts** data processing, and **makes calls** to
 - **connect** with the database server
 - **send** SQL commands to the database server
 - **fetch** tuples of result **one-by-one** into program variables

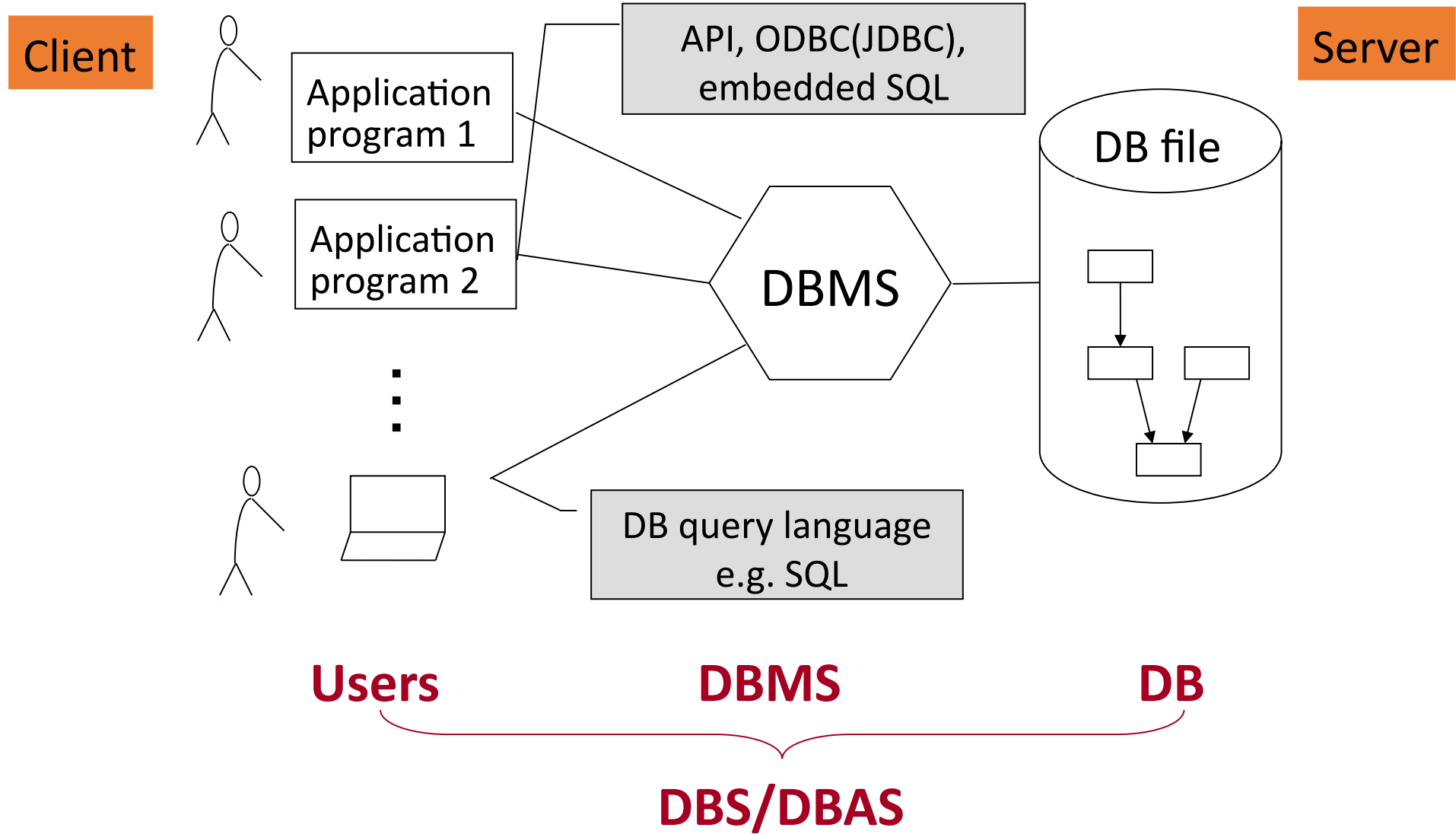


Fig.5.0.1 DBS and DBAS

Accessing SQL From a Programming Language

■ Two approaches to accessing SQL

▫ 1. dynamic SQL

- **general-purpose program**
connects to and communicate with DB server

using **functions**
- **program constructs** an SQL query as **a character string**, **submit** the query, and **retrieve** result into program variables
a tuple at a time
- **JDBC** (Java Connectivity) with Java
- **ODBC** (Open Connectivity) with C, C++, etc.

▫ 2. embedded SQL

- statements are translated in high-level programs at **compile time** into **function calls**
- **function calls** connect to DB using an **API** that provides dynamic SQL facilities

5.1.1 JDBC

- **特征** : **JDBC** is a Java-based API.
 - Supports **features** for **querying** and **updating** data, and for **retrieving** query results.
 - Supports **metadata retrieval**,
 - querying about **relations** in DB
 - querying about **names and types** of relation attributes
- **用途** : **Model** for communicating with DB:
 - **open** a connection
 - **create** a **Statement** object
 - **execute** queries using Statement to **send** queries and **fetch** results
 - **exception** mechanism to **handle** errors

```
public static void JDBCexample(String userid, String passwd)
```

```
{
    try
```

```
    {
        Class.forName("oracle.jdbc.driver.OracleDriver");
        Connection conn = DriverManager.getConnection(
            "jdbc:oracle:thin:@db.yale.edu:1521:univdb",
            userid, passwd);
```

```
        Statement stmt = conn.createStatement();
```

```
        try {
            stmt.executeUpdate(
                "insert into instructor values('77987', 'Kim', 'Physics', 98000)");
        } catch (SQLException sqle)
```

```
        {
            System.out.println("Could not insert tuple. " + sqle);
        }
```

```
        ResultSet rset = stmt.executeQuery(
            "select dept_name, avg (salary) "+
            " from instructor "+
            " group by dept_name");
```

```
        while (rset.next()) {
            System.out.println(rset.getString("dept_name") + " " +
                rset.getFloat(2));
```

```
        }
        stmt.close();
        conn.close();
```

```
    } catch (Exception sqle)
```

```
    {
        System.out.println("Exception : " + sqle);
```

```
    }
```

```
}
```

1. 选择 JDBC 驱动

动

2. 创建连接

URL/machine to be connected

3. 创建语句 stmt

4. Ship SQL query through stmt to DBMS

连接名字

Statement handle

5. 捕获异常

结果集

6. 判断是否有数据，取结果，输出

7. 关闭语句、连接

class:

DriverManager

method:

getConnection

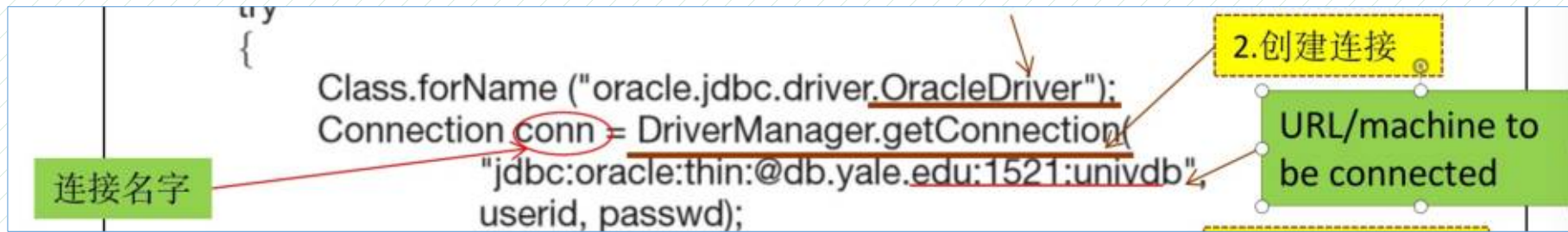
object: statement

handle: stmt

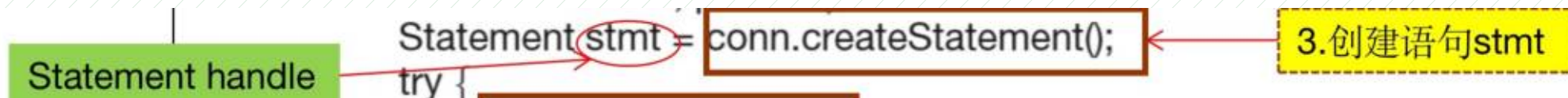
Fig.5.1 JDBC Code
Older Java/JDBC

JDBC

- **Open** a connection, named as **conn**
 - using the **getConnection** method of the DriverManager class
 - parameters
 1. URL/machine to be connected;
 2. user identifier, password;



- **Create** a SQL statement object on the connection **conn**
 - using the **createStatement** method
 - its statement handle is named as **stmt**
 - invoke methods that **ship** an SQL statement for execution




```

try {
    Class.forName ("oracle.jdbc.driver.OracleDriver");
    Connection conn = DriverManager.getConnection(
        "jdbc:oracle:thin:@db.yale.edu:2000:univdb", userid, passwd);
    Statement stmt = conn.createStatement();
    ... Do Actual Work ....
    stmt.close();
    conn.close();
}
catch (SQLException sqle) {
    System.out.println("SQLException : " + sqle);
}

```

连接名字

Statement handle

7. 关闭语句、连接

5. 捕获异常

1. 选择 JDBC 驱动程序

2. 创建连接

3. 创建语句 stmt

4. Ship SQL query through stmt to DBMS

- **NOTE:** Class.forName is **not** required from **JDBC 4** onwards.
The try with resources syntax in previous slide is for **Java 7** onwards.

- **1. Update** to database

```
try {  
    stmt.executeUpdate(  
        "insert into instructor values('77987', 'Kim', 'Physics', 98000)");  
    } catch (SQLException sqle)  
    {  
        System.out.println("Could not insert tuple. " + sqle);  
    }
```

4.Ship SQL query through stmt to DBMS

- **2. Execute** query and fetch and print results

```
ResultSet rset = stmt.executeQuery(  
    "select dept_name, avg (salary)  
    from instructor  
    group by dept_name");  
while (rset.next()) {  
    System.out.println(rset.getString("dept_name") + " " +  
        rset.getFloat(2));  
}
```

4.Ship SQL query through stmt to DBMS

结果集

6. 判断是否有数据，取结果，输出

JDBC

- **Note** : Execute queries using Statement object **stmt** to send queries and fetch results
- Using **execute.query** or **execute.update** such as insert/delete/update/createtable,
- **1. Parameters** : SQL statement to be executed, **represented as a string**
- **2. Catch** any exceptions or error conditions
- **3. Fetch** the query result , using the **try{...}/catch{...}** construct
 - **retrieve** the set of tuples in the result into a **ResultSet object rset**,
fetch them one tuple at a time
 - the **next()** method tests whether or not the result set has at least one tuple and if so, fetches it
- **4. Close** the statement **stmt** and the connection **conn** at the end

JDBC Code Details

- **Getting result fields:**
 - rs.getString("dept_name") and rs.getString(**1**) are **equivalent** if dept_name is **the first** argument of select result.
- **Dealing with Null values**

```
int a = rs.getInt("a");  
if (rs.isNull()) Systems.out.println("Got null value");
```

JDBC SUBSECTIONS

- **Connecting** to the Database
 - Shipping** SQL Statements to the Database System
 - Exceptions and Resource Management**
 - Retrieving** the Result of a Query
- 1. Prepared **Statements**
- 2. Callable **Statements**
- 3. Metadata Features

1.Prepared Statement

- **方法** : **Create a prepared statement** in which **some** values are replaced by “ ? ” thereby specifying that the actual values will be provided **later**
 - The DBS **compiles** the query when it is prepared.
 - Each time the query is executed (with new values to **replace** “?”), the DB system can **reuse** the **previously compiled form** of the query, and **apply** the new values as parameters
- **Note** : The **setString()** method, and other methods such as **setInt()** **specify the values** for the parameters
 - The **first** argument specifies the “?” parameter
The **second** argument specifies the value to be assigned
 - The first parameter is **1**, unlike most other Java constructs, which start with **0**.

Prepared Statement

an object of class

class

method

instructor(ID, name, dept name,
salary)

- PreparedStatement `pStmt` = `conn.prepareStatement(`
 `"insert into instructor values(?,?,?,?)"`);

- `pStmt.setString(1, "88877");`
 `pStmt.setString(2, "Perry");`
 `pStmt.setString(3, "Finance");`
 `pStmt.setInt(4, 125000);`
 `pStmt.executeUpdate();`

`pStmt.setString(1, "88878");`
`pStmt.executeUpdate();`

1. The first argument specifies the “?” parameter for which we are assigning a value
2. The second argument specifies the value to be assigned
 - 前 4 条语句设置 insert 的输入为：
 `values(88877, Perry, Finance, 125000)`，
 但没有触发 insert 的执行

- 将 ID 重新改为“88878”，并触发执行
 `insert into instructor values(88878, Perry, Finance, 125000)`

2. Metadata Features

- **ResultSet metadata**

- ▶ e.g. after executing query to get a ResultSet rs:

ResultSetMetaData

```
rsmd = rs.getMetaData();
```

```
for(int i = 1; i <= rsmd.getColumnCount(); i++)
```

{

```
System.out.println(rsmd.getColumnNames(i));
```

```
System.out.println(rsmd.getColumnTypeName(i));
```

}

- How is this useful?

获取查询结果集中列的数目
(结果关系的属性总数) ;
输出各列的列名、数据类型

Metadata (Cont)

- Database metadata
- DatabaseMetaData dbmd = conn.getMetaData();
// **Arguments** to getColumnns: Catalog, Schema-pattern, Table-pattern, and Column-Pattern
// **Returns**: One row for each column; row has a number of attributes
such as COLUMN_NAME, TYPE_NAME
// The value **null** indicates all Catalogs/Schemas.
// The value **""** indicates current catalog/schema
// The value **"%"** has the same meaning as SQL **like** clause
ResultSet rs = dbmd.getColumnns(null, "univdb", "department", "%");
while(rs.next()) {
 System.out.println(rs.getString("COLUMN_NAME"),
 rs.getString("TYPE_NAME");
}

Finding Primary Keys

```
■ DatabaseMetaData dmd = connection.getMetaData();  
// Arguments below are: Catalog, Schema, and Table  
// The value "" for Catalog/Schema indicates current catalog/schema  
// The value null indicates all catalogs/schemas  
ResultSet rs = dmd.getPrimaryKeys("", "", tableName);  
while(rs.next())  
{  
    // KEY_SEQ indicates the position of attribute in primary key,  
    // which is required if a primary key has multiple attributes  
    System.out.println(rs.getString("KEY_SEQ"),  
        rs.getString("COLUMN_NAME"));  
}
```

3. Transaction Control

- Each SQL statement is treated as a separate transaction that is committed automatically by default.
 - **bad idea for transactions with multiple updates**
- Turn off automatic commit on a connection
 - `conn.setAutoCommit (false);`
- Turn on automatic commit on a connection
 - `conn.setAutoCommit (ture);`
- **Note** : Transactions must then be committed or rolled back explicitly
 - `conn.commit();`
 - `conn.rollback();`

Other Features

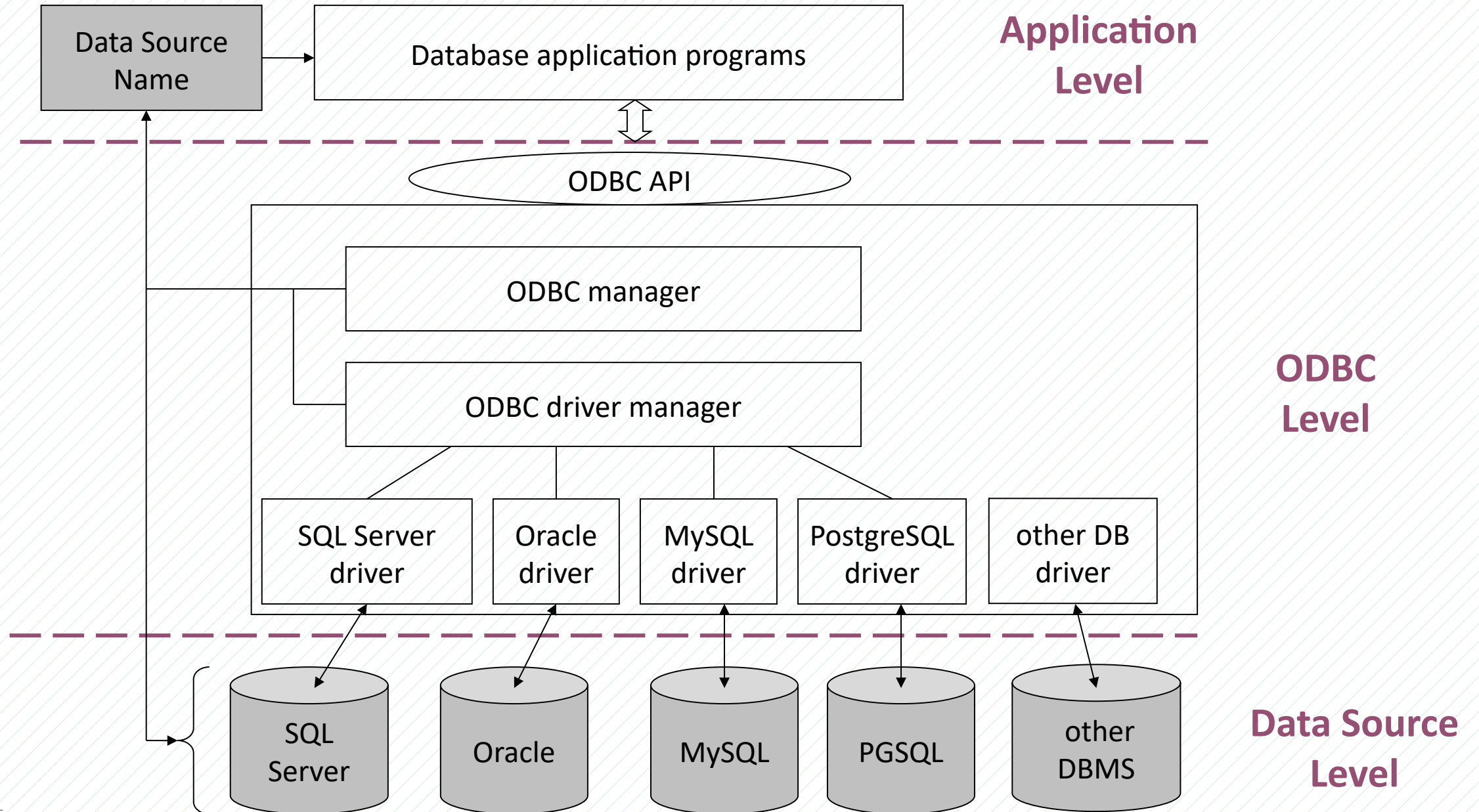
■ Calling **functions** and **procedures**

- ▣ CallableStatement cStmt1 = conn.prepareCall("{? = call some **function**(?)}");
- ▣ CallableStatement cStmt2 = conn.prepareCall("{call some **procedure**(?,?)}");

5.1.3 ODBC

- **Open DataBase Connectivity (ODBC) standard**
 - **application programs** (as clients) to communicate with DB server.
 - **application program interface (API)** to
 - **open a connection** with a database
 - **send** queries and updates
 - **get back** results

■ Architecture of ODBC



void ODBCexample()

0. Types and variables declaration

1. 分配 SQL 环境

2. 分配连接

3. 打开连接

4. 定义 C variables、SQL 语句句柄

5. 分配语句

6. Send SQL to the DBMS through statement

7. 将查询结果 tuple 中的属性绑定到 C variables

8. 从查询结果中依次取各个结果 tuple

9. 结束：
释放语句
断开连接
释放连接
释放环境

```
RETcode error;
HENV env; /* environment */
HDBC conn; /* database connection */
```

```
SQLAllocEnv(&env);
SQLAllocConnect(env, &conn);
SQLConnect(conn, "db.yale.edu", SQL_NTS, "avi", SQL_NTS,
            "avipasswd", SQL_NTS);
```

```
char deptname[80];
float salary;
int lenOut1, lenOut2;
HSTMT stmt;
```

```
char * sqlquery = "select dept_name, sum (salary)
                  from instructor
                  group by dept_name";
```

```
SQLAllocStmt(conn, &stmt);
error = SQLExecDirect(stmt, sqlquery, SQL_NTS);
if (error == SQL_SUCCESS) {
```

```
    SQLBindCol(stmt, 1, SQL_C_CHAR, deptname, 80, &lenOut1);
    SQLBindCol(stmt, 2, SQL_C_FLOAT, &salary, 0, &lenOut2);
    while (SQLFetch(stmt) == SQL_SUCCESS) {
        printf(" %s %g\n", deptname, salary);
    }
```

```
    SQLFreeStmt(stmt, SQL_DROP);
```

```
SQLDisconnect(conn);
SQLFreeConnect(conn);
SQLFreeEnv(env);
```

DBMS 执行 SQL query 后的返回码 (to client) : 成功, 失败

connection handle

SQLConnect 参数

连接句柄 : conn
连接的服务器 : db.yale.edu
用户标识 : avi
用户口令 : avipassword

statement handle

SQL_NTS: the previous argument is a NTS (null-terminated string)

Query

SQLBindCol 参数

1: 语句, e.g. stmt;
2: 查询结果 tuple 中属性位置, e.g. 1,2;
3: SQL attribute 类型 $\diamond$ C variable 类型转换;
4: the address of the C variable;
5/6: 对变长 (字符串) 类型, 定义最大长度, 及实际长度的存储位置 / 变量

ODBC code example in C program

关于连接时长

- 实际数据库平台下，client application program 与 DBMS 间连接 (connection) 的时长是有限制、并且可以调整的
- 建立连接、数据库访问时，如果不指定连接时长，可采用系统默认的连接时长
- 注意：
 - ▣ 当采用系统默认连接时长时，e.g.60s，如果在连接过程中的 SQL query 操作访问时间超过该连接时长，则 SQL query 操作将被强行终止，访问失败
 - ▣ 可以采用相关 API，修改连接时长，保证数据库访问时间不超过连接时长

5.1.4 Embedded SQL

- **Approaches** to take SQL as DB query tools
 - **1. interactive SQL**
 - Used directly as DML and DDL through DBS human-machine interfaces
 - **2. dynamic SQL**
 - e.g JDBC, ODBC
 - **3. embedded SQL**
 - Embedded in general-purpose programming languages, e.g. **C** language
 - Executing of general-purpose programming language programs with SQL statement embedded results in DB access

Embed SQL

- The SQL standard defines embeddings of SQL in languages such as C, C++, Pascal, Fortran, and Cobol
- **Def:** A language to which SQL queries are embedded is referred to as **host language(宿主语言)**.

The SQL structures permitted in host language comprise **embedded SQL**

- **Merits of embedded SQL**
 - 交互式 SQL 只能进行 DB 的访问操作，不能对 DB 访问结果进行进一步的数据处理
 - Embedded SQL 将 SQL 的数据库访问功能与 C 语言等宿主语言的数据处理能力相结合，提高了数据应用系统的能力

Embedded SQL

- **EXEC SQL** statement is used to identify **embedded SQL** request to the preprocessor

EXEC SQL <embedded SQL statement > ;

➤ Note: this varies by language:

- in some languages, like COBOL, the **semicolon** is replaced with **END-EXEC**
- in Java embedding uses **# SQL** { };

Embedded SQL (cont.)

- Before executing any SQL statements, the program must first connect to the database.

➤ 方法：

EXEC-SQL connect to *server* user *user-name* using *password*;

- **Note:** *server identifies* the server to which connection is to be established.

Embedded SQL (cont.)

- **Variables** of the **host** language can be used within embedded SQL statements. They are **preceded** by colon (:) to distinguish from SQL variables
 - e.g., **:credit_amount**
- Note : Variables used as above must be **declared** within **DECLARE** section.
- The **syntax** for **declaring the variables** follows the usual host language syntax.

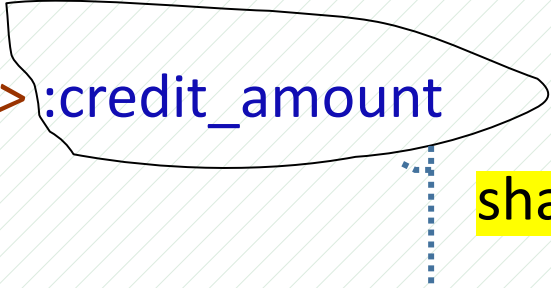
```
EXEC-SQL BEGIN DECLARE SECTION
           int credit-amount ;
EXEC-SQL END DECLARE SECTION;
```

Embedded SQL (cont.)

- **Example:** From a host language, find the **ID** and **name** of students who have completed more than the number of **credits** stored in variable **credit_amount** in the host language
 - ▣ **credit_amount** is the shared variable defined in the declaration part

■ **C program :**

```
EXEC SQL BEGIN DECLARE SECTION
        int credit_amount;
EXEC SQL END DECLARE SECTION
        credit_amount := e.g. input-from-screen-by-users
EXEC SQL
        select ID, name
        from student
        where tot_cred > :credit_amount
END-EXEC
```



shared variable defined in
host language

- To write an embedded SQL query, we use the statement :

declare *c* **cursor for** <SQL query>

- **Note** : The variable *c* is used to *identify* the query

EXEC SQL

declare *c* **cursor for**

select *ID, name*

from *student*

where *tot_cred* > :credit_amount

END_EXEC

why using Cursor in Embedded SQL

- 利用 Embedded SQL 进行查询时，查询结果有可能包括多个元组，此时无法直接将多个元组通过共享变量赋值传递给宿主程序
- 系统开辟专门 working 区域存放 SQL 查询的结果关系，并利用查询游标 **c** 指向此区域。
宿主程序根据 **c** 指向的查询结果关系集合，
使用 **open, fetch, close** 依次获取结果关系中的各元组

Example

```
EXEC SQL BEGIN DECLARE SECTION
```

```
  int credit_amount;
```

```
  char si, sn;
```

```
EXEC SQL END DECLARE SECTION
```

```
  credit_amount := input-from-screen-by-users
```

```
EXEC SQL
```

```
  declare c cursor for
```

```
    select ID, name
```

```
    from student
```

```
    where tot_cred > :credit_amount
```

```
END-EXEC
```

```
EXEC SQL open c END-EXEC
```

```
EXEC SQL fetch c into :si, :sn END-EXEC
```

```
EXEC SQL close c END-EXEC
```

可以用循环语句依次
取走全部查询结果

shared variable

Cursor in Embedded SQL (cont.)

- Usage of cursor in embedded SQL
declare cursor – open – fetch – close

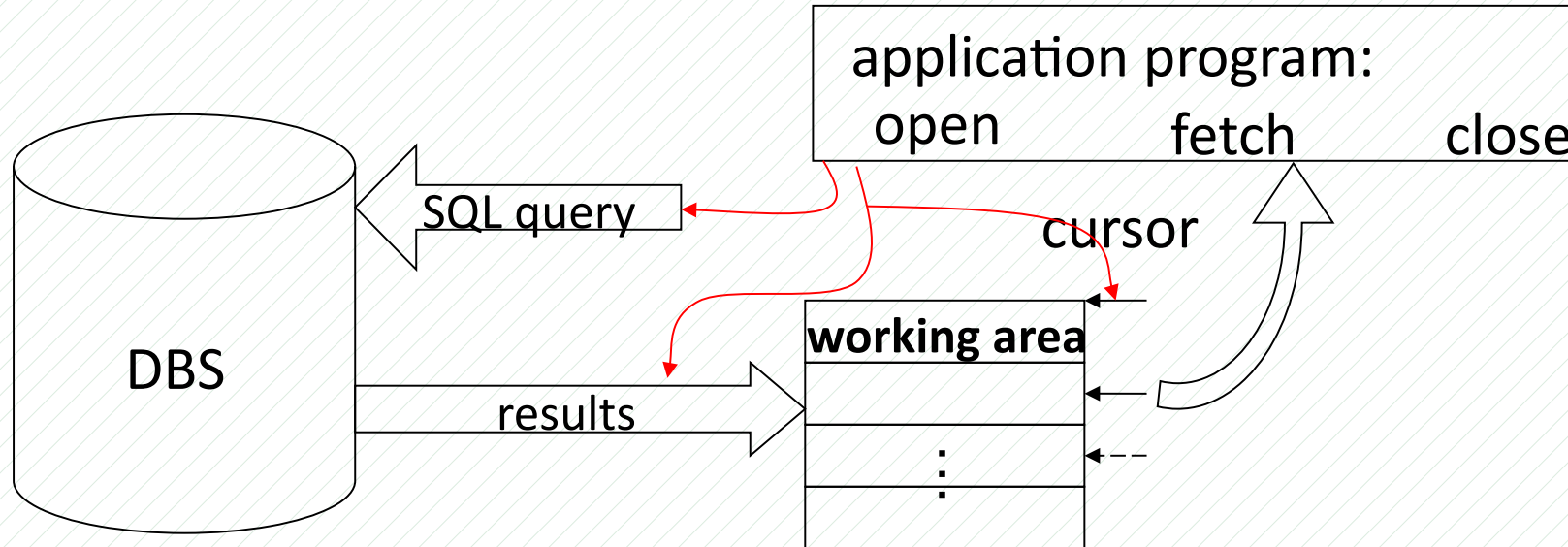


Fig. 5.0.3 Cursor in Embedded SQL

Cursor in Embedded SQL (cont.)

- The **open** statement for our example is as follows:

```
EXEC SQL open c
```

- This statement **execute** the query and **save** the results **within a temporary relation**.
- The query uses the value of the host-language variable **credit-amount** at the time the **open** statement is executed.
- The **fetch** statement causes the values of **one tuple** in the query result to **be placed on host language variables**.

```
EXEC SQL fetch c into :si, :sn END_EXEC
```

- Repeated calls to fetch get successive tuples in the query result

Cursor in Embedded SQL (cont.)

- The **close** statement **delete** the **temporary** relation that holds the result of the query

EXEC SQL close c

- **Note**
 - ▣ above details vary with language
 - ▣ the Java embedding defines *Java* iterators to step through result tuples

Updates Through Embedded SQL

- Embedded SQL expressions for **modification** (update, insert, and delete)
- Can **update** tuples **fetch**ed by cursor by **declaring that the cursor is for update**

EXEC SQL

```
declare c cursor for  
select *  
from instructor  
where dept_name = 'Music'  
for update
```

Query:

Add **1000** to the **salary**
attribute of every **instructor**
in the **Music** department

- We **iterate** through the tuples by performing **fetch** operations on the cursor,
- After fetching each tuple we execute the following code:

```
update instructor  
set salary = salary + 1000  
where current of c
```

5.2 Functions and Procedures

- SQL supports **functions** and **procedures**
 - functions/procedures can be written in **SQL** itself,
or an **external programming language** (C, Java).
 - functions written in external languages are useful with **specialized data** types such as images.
 - Example: functions to **check** if polygons overlap,
or to **compare** images for similarity,
 - DBS support **table-valued functions**,
which can return **a relation** as a result.
- SQL-99 also supports imperative constructs
 - Loops, if-then-else, assignment

Example: SQL Function

- **Function** : For the given the **name** of a department, returns the count of the number of instructors.

```
create function dept_count (dept_name varchar(20))
returns integer
begin
    declare dept_count integer;
    select count ( * ) into dept_count
    from instructor
    where instructor.dept_name = dept_name
    return dept_count;
end
```


Example: SQL Functions

- Note: The function *dept_count* can be used to find the department names and budget of all departments with more than 12 instructors

```
select dept_name, budget
from department
where dept_count (dept_name) > 12
```

SQL Procedures

- The **dept_count** function could be written as **procedure**:

```
create procedure dept_count_proc (in dept_name varchar(20),  
                                     out d_count integer)  
  
begin  
    select count(*) into d_count  
    from instructor  
    where instructor.dept_name = dept_count_proc.dept_name  
end
```

SQL Procedures

- Procedures can be **invoked** either from an SQL procedure or from embedded SQL, using the **call** statement.

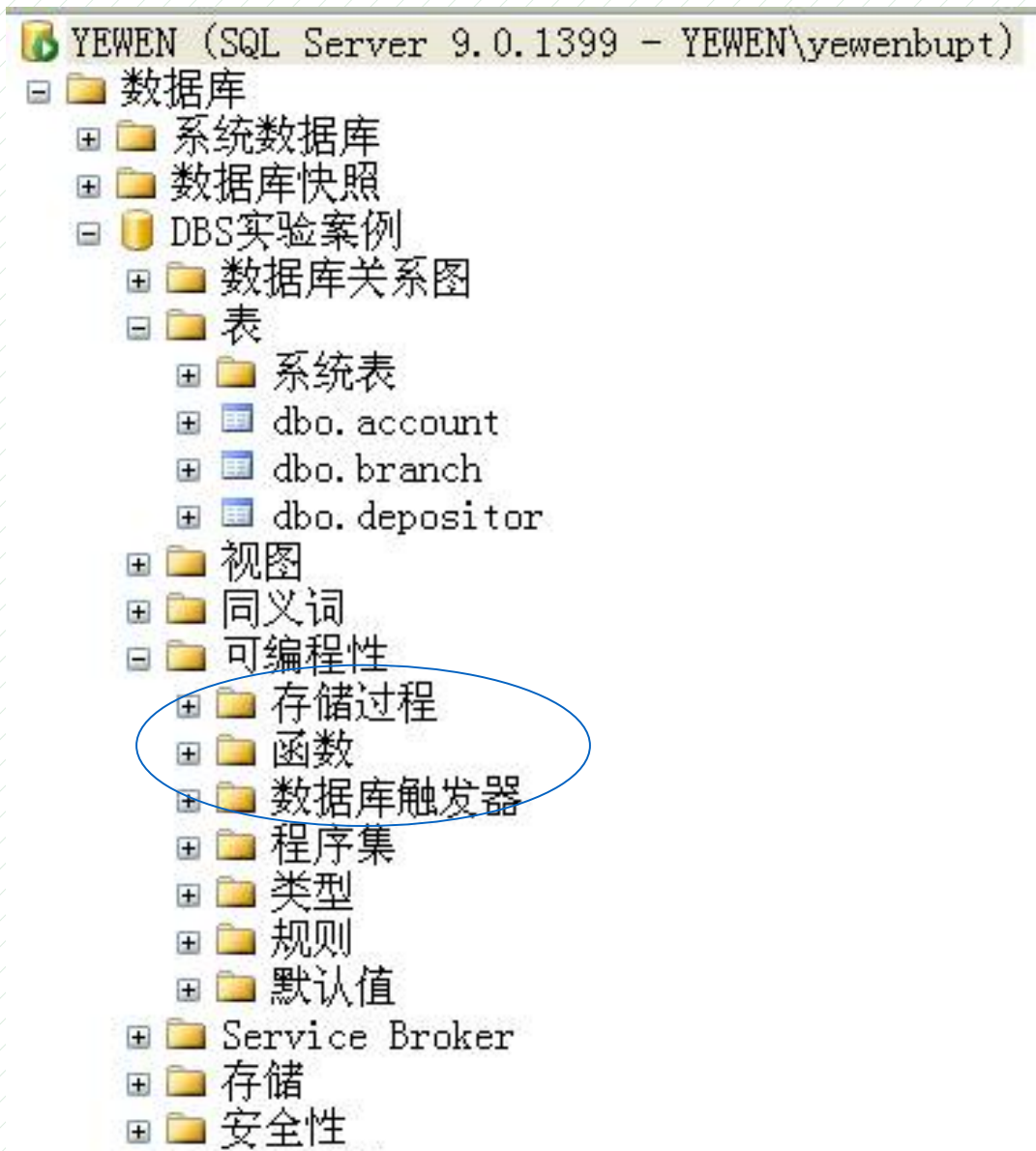
```
declare d_count integer;  
call dept_count_proc ( 'Physics', d_count );
```

- Procedures and functions can be invoked also from dynamic SQL

函数、存储过程预先生成对应的查询执行计划（类似于目标代码），存储在 DBMS 中，应用程序直接调用，不需再进行查询处理和优化（ i.e. 编译 ）

Functions and Procedures (cont.)

- The functions and procedures defined are stored in DBS,
- can be called by SQL statements or programs



5.3 Trigger (触发器)

■ Trigger

- a statement that is executed **automatically**
as a **side effect** of modification to DB

□ **Note** : As an **integrity control mechanism**, trigger introduced to SQL-99,
but supported even earlier using **non-standard** syntax.

- **syntax illustrated here may not work exactly on your DBS**

Trigger (cont.)

- **Def** : Trigger is an **event-condition-action model** based mechanism
 - **integrity definition, checking, and remedy**
 - **specify what events** cause the trigger to be executed (**insert, delete, update**)
 - **under which conditions** the trigger execution will proceed
 - integrity constraints checking
 - **specify actions** to be taken when the trigger executes
 - if **constraints is violated**, **remedy actions** are taken

Triggering Events and Actions

- **Triggering event** can be insert, delete or update
- **Note:**
 - Triggers on **update** can be restricted to specific attributes
 - e.g., after update of *takes* on **grade**
 - **Values of attributes before and after** an **update** can be referenced
 - referencing old row as : for deletes and updates
 - referencing new row as: for inserts and updates

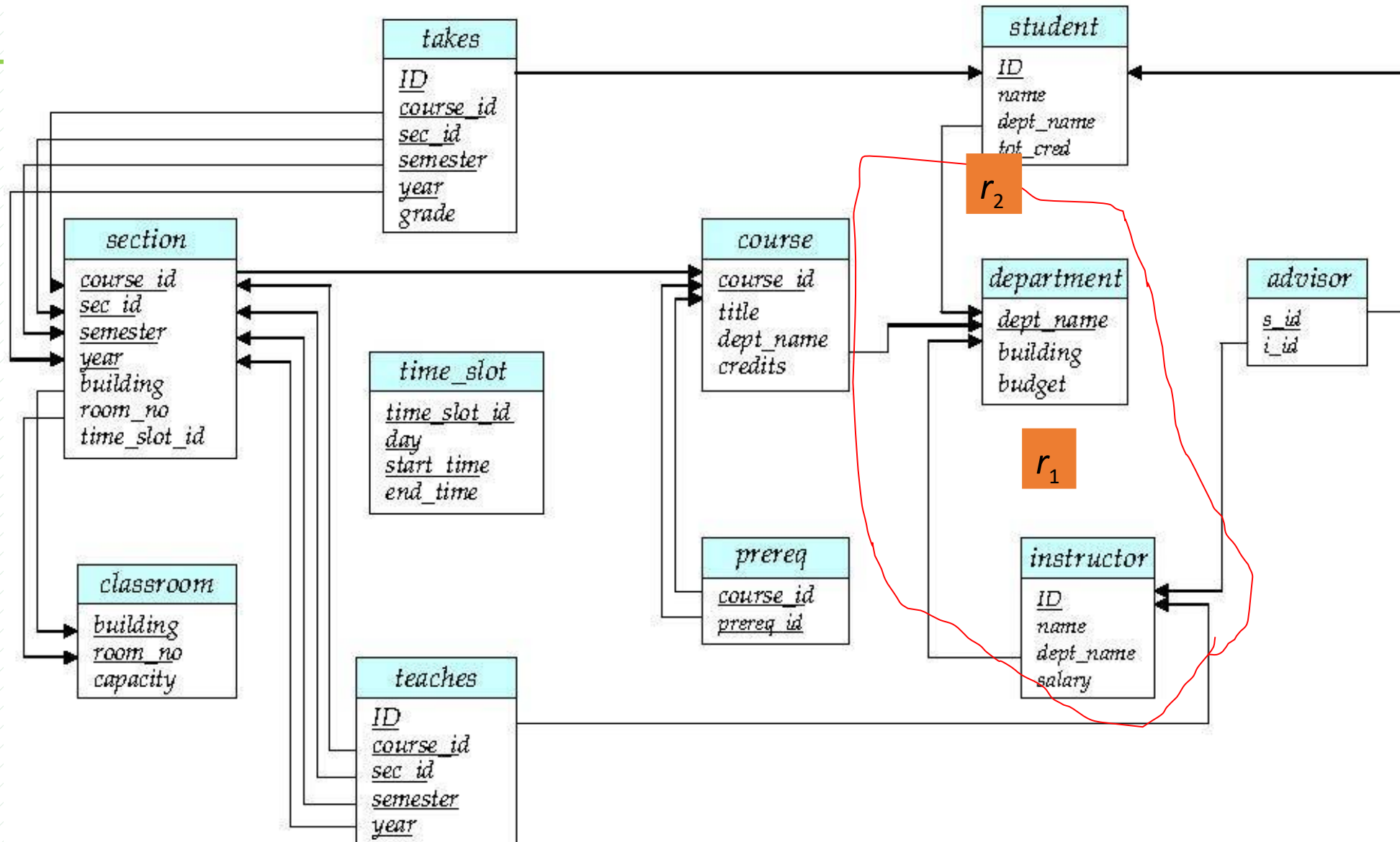


Fig. 2.8 Schema Diagram

Triggering Events and Actions

- Triggers can be activated **before** an event, which can serve as **extra constraints**.
- For example, convert **blank grades** to null.

```
create trigger setnull_trigger before update of takes
referencing new row as nrow
for each row
when (nrow.grade = ' ')
begin atomic
    set nrow.grade = null;
end;
```

当新插入元组中的成绩 grade 为 blank 时，将 grade 替换为 null，再插入。
注意：由于是 insert 操作，不存在 old row，只有 new row

优点：
预先编译（查询处理优化），存储在 DBMS 中，直接调用

Trigger to maintain referential integrity

1. 插入 section 中新元组的 time_slot_id 必须在 timeslot 中存在，否则回滚
rollback ;

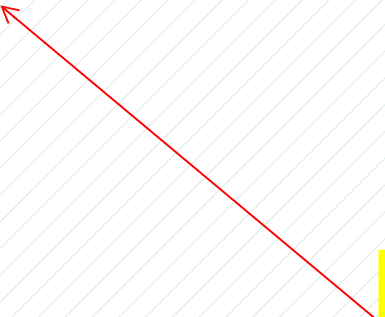
- create trigger **timeslot_check1** after insert on **section**
referencing new row as **nrow** // 插入的新元组
for each row

```
when nrow.time_slot_id not in (  
    select time_slot_id  
    from timeslot)
```

```
begin atomic  
    rollback  
end;
```

<DBS,9-12 班 , 秋季 ,2023, 教 3, 340, 3-5
节 >

新插入的 section 的
time_slot_id 还没有在 timeslot
中定义



2. 删除 *timeslot* 中某个已有元组时，该元组的 *time_slot_id* 必须在 *section* 中有，并且该元组是具有 *time_slot_id* 的唯一 / 最后一个元组；此时， **rollback**

e.g. <1-2 节, 星期二, 8:00,10:00> in

timeslot

- create trigger *timeslot_check2* after delete on *timeslot* referencing old row as **orow** // 删除前的旧元组 for each **orow** b

when *orow.time_slot_id* not in (
select *time_slot_id*

from *timeslot*) /last tuple for *time_slot_id* deleted from *timeslot*

and *orow.time_slot_id* in (
select *time_slot_id*

from *section*) /*time_slot_id* still referenced from *section*

begin atomic
rollback
end;

不存在另外一个 1-2 节上课点，
e.g. <1-2 节, 星期三, 8:00,10:00> in *timeslot*

被删除的 *timeslot* 的 *time_slot_id*，e.g. 1-2 节，在
section 中仍然使用，e.g.
<DBS,9-12 班, 秋季, 2023, 教 2, 340, 1-2 节 > in
section

Example: 数据导入触发器

例 3. 小学期课程设计

- 数据导入时，利用触发器对导入数据库的数据进行预处理：
 - ▣ 1) 新数据 insert ；
 - ▣ 2) 已有数据： update ，或删除旧数据后，再插入新数据

Example: Trigger to Maintain credits_earned value

当通过 **update** takes 向 takes 中填入成绩时，要求更新 student 表中学生的

tot_credits
• **create trigger credits_earned after update of takes on (grade)**

referencing new row as nrow // 修改后新成绩元组

referencing old row as orow // 修改前旧成绩元组

for each row

when nrow.grade <> 'F' and nrow.grade is not null

and (orow.grade = 'F' or orow.grade is null)

begin atomic

update student

set tot_cred= tot_cred +

(select credits

from course

where course.course_id= nrow.course_id)

where student.id = nrow.id;

end;

update 之后 takes 中该门课及格并且有成绩◇ **新修完一门课程，并且获得新学分，需要在 student.tot_credits 中累加新学分**

update 之前 takes 中该门课考试没有通过或没有成绩

被 update 的学
生

被 update 的课
程

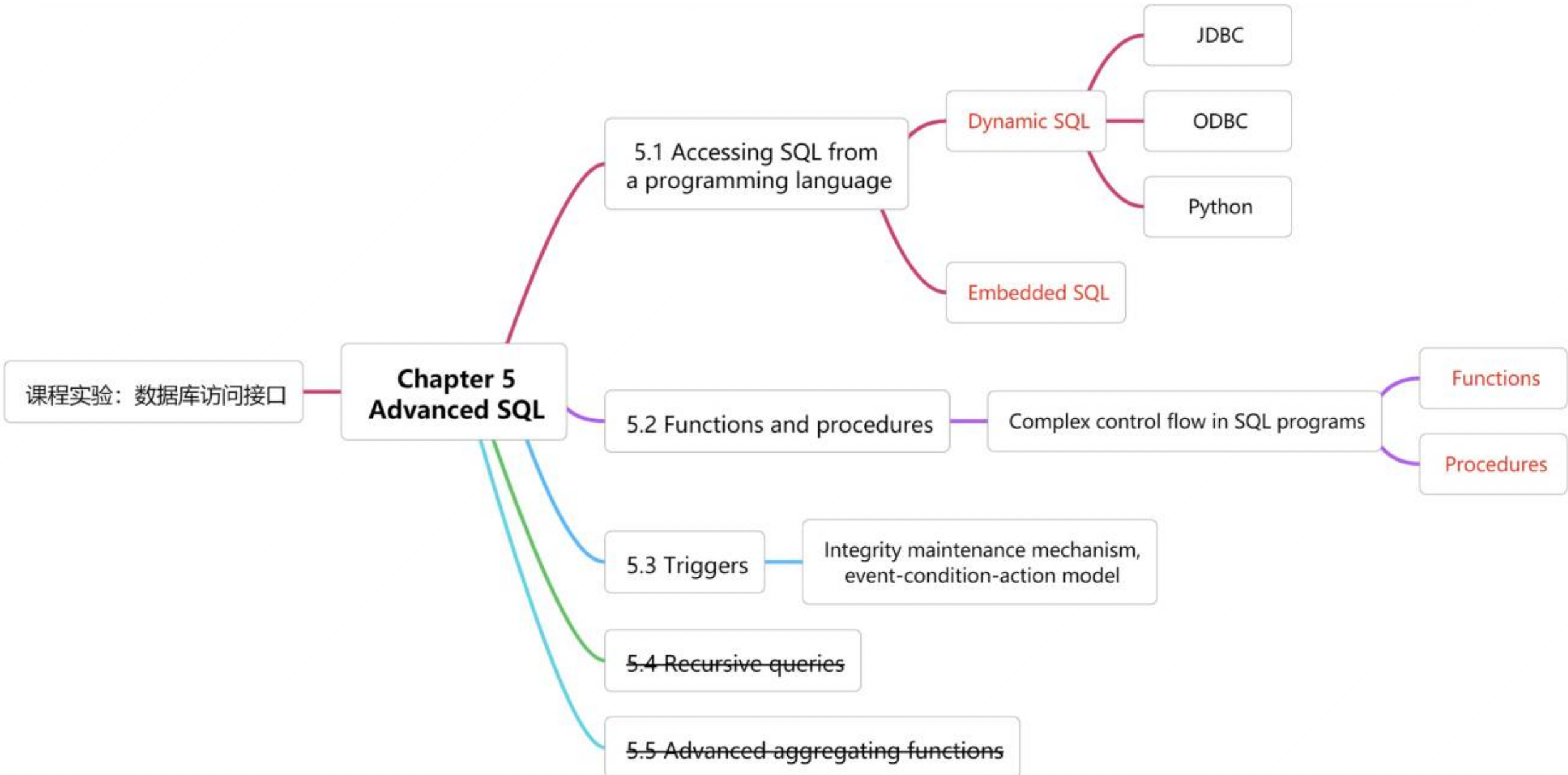
Statement Level Triggers

- Instead of **executing** a separate action for **each affected row**, a **single action** can be executed for **all rows** affected by a transaction
 - **Use** for each statement instead of for each row
 - **Use** referencing old table or referencing new table to refer to temporary tables (called *transition tables*) containing the affected rows
 - It is more efficient when dealing with **update** a large number of rows

When Not To Use Triggers

- Triggers were used earlier for tasks such as
 - ▣ Maintaining summary data (e.g., total salary of each department)
 - ▣ Replicating databases by recording changes to special relations (called **change** or **delta** relations) and having a separate process that applies the changes over to a replica
- There are better ways of doing these now:
 - ▣ Databases today provide built-in materialized view facilities to maintain summary data
 - ▣ Databases provide built-in support for replication
- Encapsulation facilities can be used instead of triggers in many cases
 - ▣ Define methods to update fields
 - ▣ Carry out actions as part of the update methods instead of through a trigger

Conclusions





Thanks for your
attention